ИТМО

Компонентный frontend и современные фреймворки

Подходы к реализации интерфейса

ИТМО

Императивный подход

- Ручной поиск/создание/удаление элементов
- Ручная обработка событий
- Ручное управление состояниями
- Ручная изоляция слоев
- Все сами…

Декларативный подход



- Автоматическое изменение интерфейса при изменении данных
- Готовый набор слушателей, только сделай обработчик
- Изоляция состояний внутри компонентов/набора компонентов, передача между родственниками
- Компоненты как слои

Императивный vs декларативный

ИТМО

```html
<p id="counter">Количество: 0</p>
<button id="increment">Увеличить</button>

<script>
  let count = 0;

  const counter =
    document.getElementById('counter');
  const button =
    document.getElementById('increment');

  button.addEventListener('click', () => {
    count += 1;
    counter.textContent = `Количество: ${count}`;
  });
</script>
```

Количество: 0

Увеличить

```jsx
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Количество: {count}</p>
      <button
        onClick={() => setCount(previous => previous + 1)}
      >
        Увеличить
      </button>
    </div>
  );
}
```

Количество: 0

Увеличить

Вспоминаем SPA компоненты

Пример Angular-компонента

Компонент - независимый переиспользуемый блок интерфейса, из которых состоит SPA.

Обычно является совокупностью шаблона (HTML-документ), локальных стилей (CSS) и класса, обслуживающего шаблон (JS/TS)

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'example';
}
```

UI = Component(props, state)

Props

ИТМО

Props - входные данные компонента, передаваемые родителем

- Строго сверху вниз
- Не должны меняться дочерним компонентом
- Описывают внешний контракт компонента

```
// Greeting.svelte
<script>
  let { name = 'Гость' } = $props();
</script>

<p>Привет, {name}!</p>

// App.svelte
<script>
  import Greeting from './Greeting.svelte';
</script>

<Greeting name="Анна" />
<Greeting name="Иван" />
<Greeting />
```

Callback props

ИТМО

```vue
// Greeting.vue
<script setup>
defineProps({
  name: String,
  onGreet: Function,
});
</script>

<template>
  <button @click="onGreet(name)">
    Поздороваться с {{ name }}
  </button>
</template>
```

```vue
// App.vue
<script setup>
import { ref } from 'vue';
import Greeting from './Greeting.vue';

const message = ref('');

function handleGreet(name) {
  message.value = `Привет, ${name}!`;
}
</script>

<template>
  <Greeting name="Анна" :on-greet="handleGreet" />
  <p>{{ message }}</p>
</template>
```

Props drilling

```
// App.svelte
<script>
  import Header from './Header.svelte';

  let name = 'Анна';
</script>

<Header {name} />

// Header.svelte
<script>
  import UserInfo from './UserInfo.svelte';

  let { name } = $props();
</script>

// UserInfo.svelte
<header>
  <h1>Мой сайт</h1>
  <UserInfo {name} />
</header>

<script>
  let { name } = $props();
</script>

<p>Пользователь: {name}</p>
```

Плохо, если:

- Промежуточных компонентов слишком много
- Данные нужны в слишком большом количестве мест
- Изменение аффектит слишком много веток

Slots

ИТМО

Slots - точки расширения внутри шаблона, куда родительский компонент способен вставлять HTML-код/другие компоненты

- Использование дочернего компонента как каркаса
- Гибкость компоновки
- Переиспользование кода

```
<Modal>
  <h2>Подтверждение</h2>
  <p>Удалить объект?</p>
</Modal>
```

Принципы хорошего компонента

ИТМО

```
Плохо:
ApplicationEverythingCompo
nent

Хорошо:
UserList
UserCard
UserAvatar
UserActions
```

Ограниченная ответственность

- Отображает в шаблоне, получает через props и хранит в состоянии только необходимое
- Иначе - разделяй/рефактори

```
Плохо:

<UserCard state="...">


Хорошо:

<UserCard userInfo="...">
```

Явный контракт

- Какие данные принимает компонент?
- Какие события он порождает?
- Что отображает?
- Какие состояния имеет?

```
Плохо:

UserCard => InfoPanelA

AdminCard => InfoPanelB


Хорошо:

UserCard => InfoPanel

AdminCard => InfoPanel
```

Переиспользуемость

- Уменьшение сложности
- Изоляция состояния
- Повышение читаемости
- Разделение ответственности

И правильная иерархия!

Реактивность

ИТМО

Реактивность - механизм автоматического обновления интерфейса при изменении данных

- Декларативность
- Скорость разработки
- Увеличение потребления памяти

У каждого фреймворка своя реализация, но есть и сторонние (RxJS)

Фреймворк	Реализация
Angular	signals + RxJS + Change Detection
React	hooks + Virtual DOM
Vue	ref/reactive
Svelte	runes

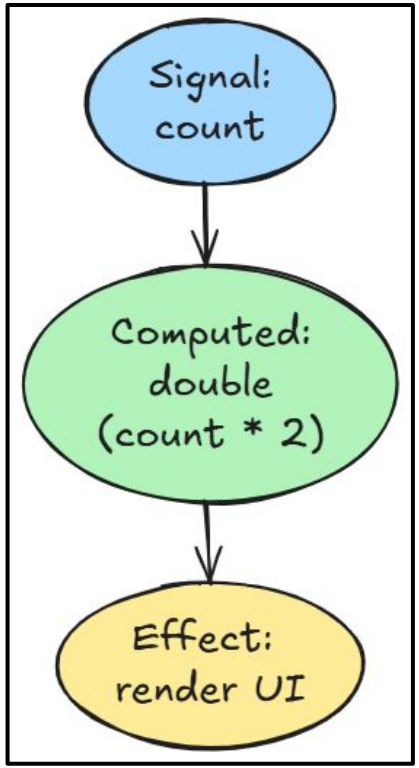
Dependency tracking

ИТМО

Отслеживание зависимостей - механизм автоматического связывания вычисления данных и участков кода/шаблона, зависящих от этих данных.

Жизненный цикл ограничен ЖЦ компонента, при его уничтожении - очистка графа

Строится в процессе выполнения кода

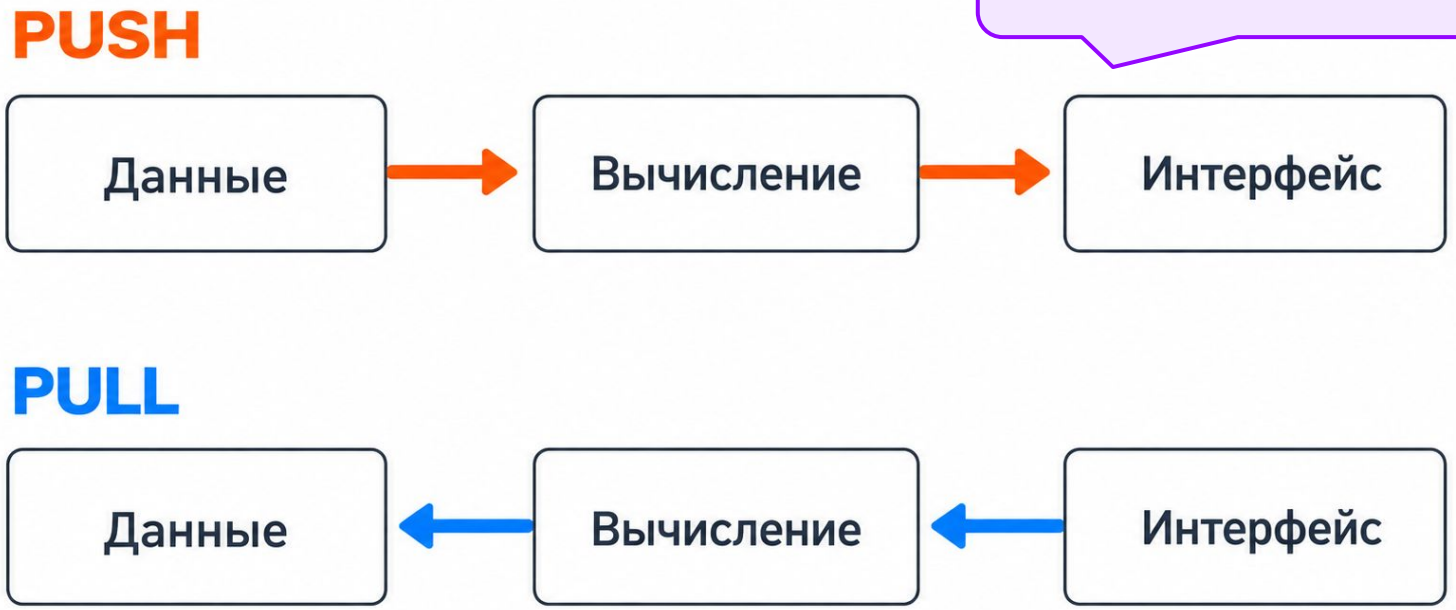


Граф зависимости

Модели реактивности

ИТМО

Работают по принципу Producer-Consumer



Реактивность

ИТМО

```
<script setup>
let count = 0;

function increment() {
  count++;
  console.log(count); // Значение увеличивается
}
</script>

<template>
  <p>{{ count }}</p>
  <button @click="increment">+1</button>
</template>
```

Какой будет вывод консоли и что покажет шаблон?

Реактивность

ИТМО

```
<script setup>
let count = 0;

function increment() {
  count++;
  console.log(count); // Значение увеличивается
}
</script>

<template>
  <p>{{ count }}</p>
  <button @click="increment">+1</button>
</template>
```

Переменная не ref

1
2
3

0

Computed properties

ИТМО

```vue
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const doubled = computed(() => count.value * 2);
</script>

<template>
  <p>Счётчик: {{ count }}</p>
  <p>Удвоенное значение: {{ doubled }}</p>

  <button @click="count++">+1</button>
</template>
```

Какой будет вывод консоли и что покажет шаблон?

Computed properties

ИТМО

```vue
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const doubled = computed(() => count.value * 2);
</script>

<template>
  <p>Счётчик: {{ count }}</p>
  <p>Удвоенное значение: {{ doubled }}</p>

  <button @click="count++">+1</button>
</template>
```

На каждое изменение count doubled пересчитывается

count: 1, 2, 4...
doubled: 2, 4, 8...

Watcher/Effect

ИТМО

Watchers

Конструкция, выполняющая некоторый side-эффект при изменении явно объявленных источников:

- Исполнение только при изменении объявленных зависимостей
- Игнорирование остальных

Effects



Конструкция, выполняющая некоторый side-эффект при изменении используемых внутри источников:

- Отслеживание всех используемых внутри зависимостей
- Исполнение при изменении любого из них

Обе отвечают только за побочное действие!

Watcher/Effect

ИТМО

```
effect(() => {
  console.log(count());
});
```

```
watch(count, (newValue, oldValue)
=> {
  console.log(`Было ${oldValue},
стало ${newValue}`);
});
```

Оптимизации реактивности

ИТМО

Compiler-based reactivity

Подход к компиляции реактивных участков кода, включающий оптимизацию DOM-операций, актуален для Svelte.

1. Во время сборки компилятор определяет зависимости
2. Компилятор внедряет инструкции отслеживания изменений/изменения DOM
3. Profit

Fine-grained reactivity

Подход к управлению состоянием, когда происходит обновление конкретного текстового узла в рамках компонента, а не всего компонента.

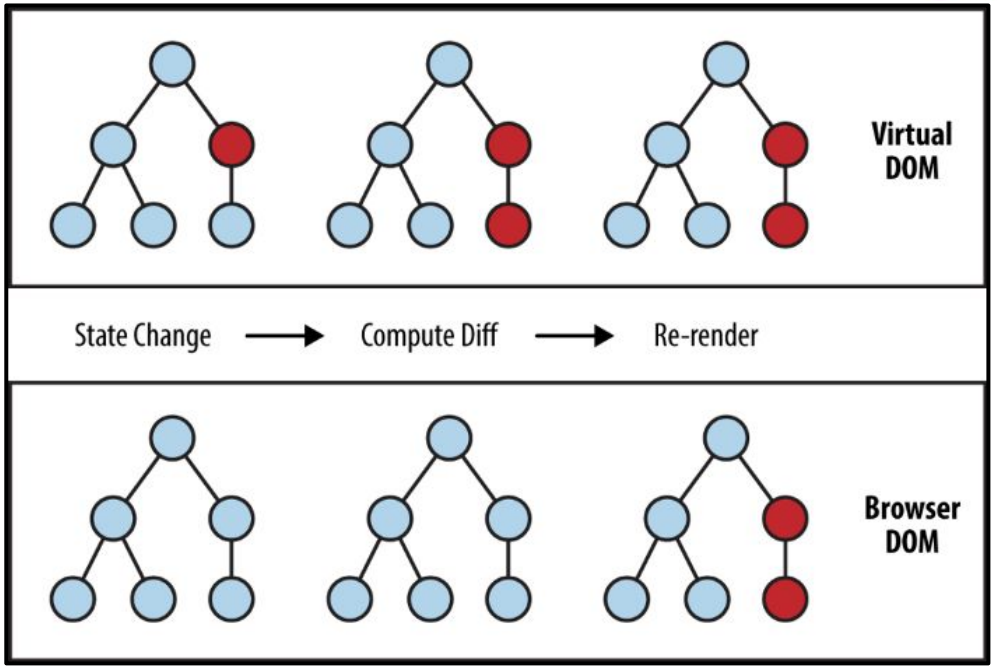
- Реализуется с помощью сигналов, вычисляемых состояний, побочных эффектов

Virtual DOM

ИТМО

Виртуальный DOM - объектное представление интерфейса.

- Точечные изменения DOM
- Уменьшение числа перерисовок
- Занимает оперативную память



Angular

ИТМО

Что это

Angular - фреймворк для создания одностраничных приложений на языке TS. Разработан компанией Google (AngularJS), на текущий момент open-source.

Особенности

- Встроенный набор инструментов
- TypeScript
- Angular CLI
- Компонентная архитектура

Angular component

ИТМО

Декоратор - спец. функция, добавляющая метаданные

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'example';
}
```

Селектор компонента в шаблоне

Ассоциированные с компонентом стили и шаблон

Angular module

ИТМО

Модуль - контейнер для изоляции связанных компонентов.

- Объединение связанных компонентов
- Инкапсуляция компонентов
- Ленивая загрузка
- Подключение зависимостей

```
import { Component, NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { GreetingModule } from './greeting.module';

@Component({
  selector: 'app-root',
  standalone: false,
  template: `<app-greeting name="Анна"></app-greeting>`,
})
export class AppComponent {}

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, GreetingModule],
  bootstrap: [AppComponent],
})
export class AppModule {}
```



а еще exports и providers

Angular service

ИТМО

Определяется, на каком уровне будет зарегистрирован

```
import { Injectable } from '@angular/core'

@Injectable({
  providedIn: 'root'
})
export class AuthorizationService {

  constructor() { }

  isLoggedJudge = true;
  isLoggedTimekeeper = false;
```

- root - в корневом уровне, 1 на все приложение
- <имя модуля> - внутри указанного модуля и всех его потомков
- any - экземпляр на каждый запрашиваемый модуль

Angular DI

ИТМО

```ts
// greeting.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class GreetingService {
  getGreeting(name: string): string {
    return `Привет, ${name}!`;
  }
}

// app.component.ts
import { Component, inject } from '@angular/core';
import { GreetingService } from './greeting.service';

@Component({
  selector: 'app-root',
  standalone: true,
  template: `<p>{{ message }}</p>`,
})
export class AppComponent {
  private greetingService = inject(GreetingService);

  message = this.greetingService.getGreeting('Анна');
}
```

Можно и через конструктор

Компонент запрашивает зависимость → Angular ищет ее в Инжекторах начиная с самого маленького окружения и идет вверх по родителям → При нахождении возвращает объект.

- NullInjector
- EnvironmentInjector
- ElementInjector

Структурные директивы

ИТМО

```ts
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-user-role',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div [ngSwitch]="role">
      <p *ngSwitchCase="'admin'">Панель администратора</p>
      <p *ngSwitchCase="'student'">Кабинет студента</p>
      <p *ngSwitchCase="'teacher'">Кабинет преподавателя</p>
      <p *ngSwitchDefault>Неизвестная роль</p>
    </div>
  `,
})
export class UserRoleComponent {
  role = 'student';
}
```

```ts
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-students',
  standalone: true,
  imports: [CommonModule],
  template: `
    <button (click)="showStudents = !showStudents">
      Показать / скрыть студентов
    </button>

    <ul *ngIf="showStudents">
      <li *ngFor="let student of students">
        {{ student }}
      </li>
    </ul>
  `,
})
export class StudentsComponent {
  showStudents = false;

  students = ['Анна', 'Иван', 'Мария'];
}
```

Angular Input

ИТМО

```ts
// greeting.component.ts
@Component({
  selector: 'app-greeting',
  standalone: true,
  template: `
    <p>Привет, {{ name }}!</p>
    <button (click)="onGreet(name)">Поздороваться</button>
  `,
})
export class GreetingComponent {
  @Input() name = '';
  @Input() onGreet: (name: string) => void = () => {};
}
```

```ts
// app.component.ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [GreetingComponent],
  template: `
    <app-greeting
      [name]="userName"
      [onGreet]="handleGreet"
    />
    <p>{{ message }}</p>
  `,
})
export class AppComponent {
  userName = 'Анна';
  message = '';

  handleGreet = (name: string) => {
    this.message = `Пользователь ${name} нажал кнопку`;
  };
}
```

Angular Output

ИТМО

```ts
// greating.component.ts
@Component({
  selector: 'app-greeting',
  standalone: true,
  template: `
    <button (click)="greet.emit(name())">
      Поздороваться с {{ name() }}
    </button>
  `,
})
export class GreetingComponent {
  name = input('');
  greet = output<string>();
}
```

```ts
// app.component.ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [GreetingComponent],
  template: `
    <app-greeting
      [name]="userName"
      (greet)="handleGreet($event)"
    />
    <p>{{ message }}</p>
  `,
})
export class AppComponent {
  userName = 'Анна';
  message = '';

  handleGreet(name: string) {
    this.message = `Привет, ${name}!`;
  }
}
```

Angular template

ИТМО

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [NgTemplateOutlet],
  template: `
    <!-- Объявляем шаблон -->
    <ng-template #userCard let-name="name">
      <div class="card">
        <p>Пользователь: {{ name }}</p>
      </div>
    </ng-template>

    <!-- Выводим шаблон с разными данными -->
    <ng-container
      [ngTemplateOutlet]="userCard"
      [ngTemplateOutletContext]="{ name: 'Анна' }"
    />
  `,
})
export class AppComponent {}
```

Angular signals

ИТМО

```
@Component({
  selector: 'app-counter-controls',
  standalone: true,
  template: `
    <p>В дочернем: {{ count()() }}</p>
    <button (click)="increment()">+1</button>
  `,
})
export class CounterControls {
  count = input.required<WritableSignal<number>>();

  increment() {
    this.count().update(value => value + 1);
  }
}
```

Используется как props

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterControls],
  template: `
    <p>Счётчик: {{ count() }}</p>
    <p>Удвоенное значение: {{ doubled() }}</p>

    <app-counter-controls [count]="count" />
  `,
})
export class AppComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);
}
```

Angular CLI

ИТМО

Angular CLI - инструмент для работы с Angular приложением.

- Сборка
- Развертывание
- Тестирование
- Инициализация проекта
- Создание компонентов

```
ng new <имя>

ng generate <сущность> <имя>

ng test

ng serve

ng build
```

React

ИТМО

Что это

React - декларативная библиотека для создания одностраничных приложений на языке TS. Разработана компанией [ДАННЫЕ УДАЛЕНЫ], на текущий момент open-source.

Особенности

- JSX
- Виртуальный DOM
- Хуки
- Компонентная архитектура

React component

ИТМО

```jsx
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Счётчик: {count}</p>
      <p>Удвоенное значение: {count * 2}</p>

      <button onClick={() => setCount(prev => prev + 1)}>
        +1
      </button>
    </div>
  );
}
```

```jsx
import { Component } from 'react';

export default class Counter extends Component {
  state = {
    count: 0,
  };

  increment = () => {
    this.setState(previousState => ({
      count: previousState.count + 1,
    }));
  };

  render() {
    const { count } = this.state;

    return (
      <div>
        <p>Счётчик: {count}</p>
        <p>Удвоенное значение: {count * 2}</p>

        <button onClick={this.increment}>
          +1
        </button>
      </div>
    );
  }
}
```

JSX

ИТМО

JSX - расширение синтаксиса JS для встраивания HTML разметки. Транспилируется в обычный JS

- Наглядность и декларативность
- Разработка быстрее
- Экранирование любого значения перед рендерингом

```jsx
export default function Greeting() {
  const name = 'Анна';

  return (
    <div className="greeting">
      <h1>Привет, {name}!</h1>
      <p>Сегодня хороший день для изучения React.</p>
    </div>
  );
}
```

JSX

ИТМО

```jsx
import { useState } from 'react';

const students = [
  { id: 1, name: 'Анна', passed: true },
  { id: 2, name: 'Иван', passed: false },
  { id: 3, name: 'Мария', passed: true },
];

function StudentItem({ student, onSelect }) {
  return (
    <li className={student.passed ? 'passed' : 'pending'}>
      <button onClick={() => onSelect(student)}>
        {student.name}
      </button>

      <span>
        {student.passed ? ' — зачёт' : ' — нет зачёта'}
      </span>
    </li>
  );
}
```

```jsx
export default function StudentList() {
  const [query, setQuery] = useState('');
  const [selected, setSelected] = useState(null);

  const filtered = students.filter(student =>
    student.name.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <>
      <h1>Студенты</h1>
      <input
        aria-label="Поиск студентов"
        placeholder="Введите имя"
        value={query}
        onChange={event => setQuery(event.target.value)}
      />
      {filtered.length > 0 ? (
        <ul>
          {filtered.map(student => (
            <StudentItem
              key={student.id}
              student={student}
              onSelect={setSelected}
            />
          ))}
        </ul>
      ) : (
        <p>Никого не найдено</p>
      )}
      {selected && <p>Выбран студент: {selected.name}</p>}
    </>
  );
}
```

props

ИТМО

```jsx
function Greeting({ name, age }) {
  return (
    <p>
      Привет, {name}! Тебе {age} лет.
    </p>
  );
}

export default function App() {
  return <Greeting name="Анна" age={20} />;
}
```

```jsx
import { Component } from 'react';

class Greeting extends Component {
  render() {
    const { name, age } = this.props;

    return (
      <p>
        Привет, {name}! Тебе {age} лет.
      </p>
    );
  }
}

export default function App() {
  return <Greeting name="Анна" age={20} />;
}
```

Callback props

ИТМО

```
function Button({ onHello }) {
  return <button onClick={onHello}>Поздороваться</button>;
}

export default function App() {
  function sayHello() {
    alert('Привет!');
  }

  return <Button onHello={sayHello} />;
}
```

onSomething - соглашение об именовании CP

slots

ИТМО

```jsx
function Card({ children }) {
  return <section className="card">{children}</section>;
}

<Card>
  <h2>Заголовок</h2>
  <p>Содержимое карточки</p>
</Card>
```

Можно передавать шаблон через props:
<Card header={<h2>...</h2>}>

React hooks

ИТМО

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(value => value + 1)}>
      Нажато: {count}
    </button>
  );
}
```

хуки - функции, позволяющие функциональным компонентам использовать состояния, эффекты и др

- useState - сохранение состояния между рендерами
- useEffect - побочные эффекты

Все хуки имеют свои аналоги в компоненте-классе

React hooks

ИТМО

```jsx
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  // По принципу componentDidMount и componentDidUpdate:
  useEffect(() => {
    // Обновляем заголовок документа, используя API браузера
    document.title = `Вы нажали ${count} раз`;
  });

  return (
    <div>
      <p>Вы нажали {count} раз</p>
      <button onClick={() => setCount(count + 1)}>
        Нажми на меня
      </button>
    </div>
  );
}
```

React hooks

ИТМО

```jsx
import { createContext, useContext } from 'react';

const ThemeContext = createContext('light');

function Button() {
  const theme = useContext(ThemeContext);

  return <button className={theme}>Сохранить</button>;
}

export default function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Button />
    </ThemeContext.Provider>
  );
}
```

Хук передачи контекста компоненту напрямую

Custom hook

ИТМО

```jsx
import { useState } from 'react';

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  function increment() {
    setCount(value => value + 1);
  }

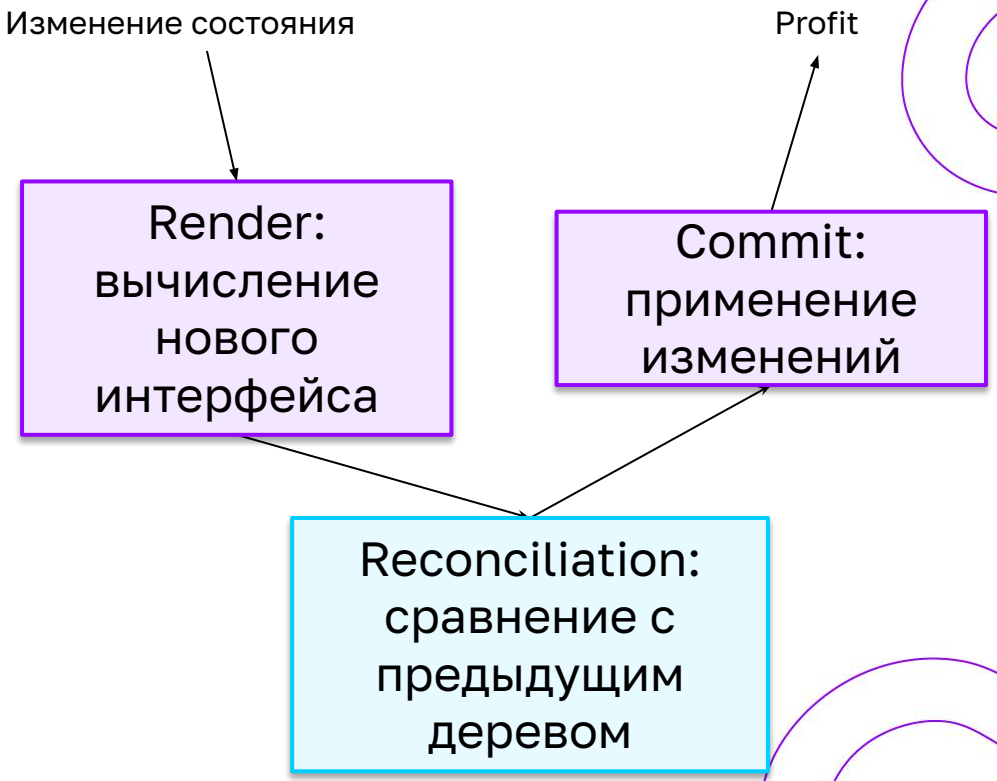
  function reset() {
    setCount(initialValue);
  }

  return { count, increment, reset };
}
```

```jsx
function Counter() {
  const { count, increment, reset } = useCounter();

  return (
    <>
      <p>{count}</p>
      <button onClick={increment}>+1</button>
      <button onClick={reset}>Сбросить</button>
    </>
  );
}
```

React reconciliation



Процесс сопоставления и выделения различий между старой и новой версиями интерфейсов

Узел пересобирается, если:

- Изменился тип элемента
- Изменился key

Vue.js

ИТМО

Что это

Vue.js (Vue) - прогрессивный JS-фреймворк для создания SPA. Разработан Эваном Ю в 2014 году.

Особенности

- Декларативность
- Виртуальный DOM
- Встроенный набор инструментов
- Composition API

Vue component

ИТМО

script - логика компонента

```
<script setup>
const name = 'Анна';
</script>

<template>
  <p class="greeting">Привет, {{ name }}!</p>
</template>

<style scoped>
.greeting {
  color: blue;
}
</style>
```

template - шаблон

style - ну вы поняли...

Реактивность

ИТМО

```vue
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const doubled = computed(() => count.value * 2);
</script>

<template>
  <p>Счётчик: {{ count }}</p>
  <p>Удвоенное значение: {{ doubled }}</p>

  <button @click="count++">+1</button>
</template>
```

Props

ИТМО

```vue
<script setup>
const props = defineProps({
  name: {
    type: String,
    default: 'Гость',
  },
});
</script>

<template>
  <p>Привет, {{ props.name }}!</p>
</template>
```

```vue
<Greeting name="Анна" />
<Greeting :name="userName" />
```

Callback props

ИТМО

```vue
// Greeting.vue
<script setup>
defineProps({
  name: String,
  onGreet: Function,
});
</script>

<template>
  <button @click="onGreet(name)">
    Поздороваться с {{ name }}
  </button>
</template>
```

```vue
// App.vue
<script setup>
import { ref } from 'vue';
import Greeting from './Greeting.vue';

const message = ref('');

function handleGreet(name) {
  message.value = `Привет, ${name}!`;
}
</script>

<template>
  <Greeting name="Анна" :on-greet="handleGreet" />
  <p>{{ message }}</p>
</template>
```

Emits

ИТМО

```vue
<!-- Greeting.vue -->
<script setup>
const emit = defineEmits(['greet']);
</script>

<template>
  <button @click="emit('greet', 'Анна')">
    Поздороваться
  </button>
</template>

<Greeting @greet="name => console.log(`Привет, ${name}!`)" />
```

Подпись на событие, а не вызов функции, поэтому результат обработки потомку неизвестен

Slots

ИТМО

```vue
<!-- Layout.vue -->
<template>
  <header><slot name="header" /></header>
  <main><slot /></main>
</template>

<Layout>
  <template #header>
    <h1>Мой сайт</h1>
  </template>

  <p>Основное содержимое</p>
</Layout>
```

Именованные слоты

Scoped слоты

```vue
<!-- UserProvider.vue -->
<template>
  <slot :name="'Анна'" />
</template>

<UserProvider v-slot="{ name }">
  <strong>{{ name }}</strong>
</UserProvider>
```

Директивы

ИТМО

v-bind, или :	Динамическая привязка атрибута/prop	:disabled="loading"
v-on, или @	Обработчик события	@click="save"
v-if	Условное создание элемента	v-if="visible"
v-else-if, v-else	Альтернативные ветки	v-else
v-show	Переключение CSS-свойства display	v-show="visible"
v-for	Вывод списка	v-for="user in users"
v-model	Согласование значения и его изменений	v-model="name"

И много других...

Composition API

ИТМО

Набор функций для организации логики работы компонента. До Vue3 - Options API

Включает lifecycle хуки:

- onMounted - компонент смонтирован, его DOM доступен
- onUpdated - DOM компонента обновлён
- onUnmounted - компонент удалён

```
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const doubled = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```

lifecycle hooks

ИТМО

```vue
<script>
import { onMounted, onUnmounted } from 'vue';

let timerId;

onMounted(() => {
  timerId = setInterval(() => console.log('Тик'), 1000);
});

onUnmounted(() => {
  clearInterval(timerId);
});
</script>
```

Options API

ИТМО

```
<script>
export default {
  data() {
    return { count: 0 };
  },

  methods: {
    increment() {
      this.count++;
    },
  },
};
</script>

<template>
  <button @click="increment">{{ count }}</button>
</template>
```

Vue composable

ИТМО

Функция, использующая Composition API для инкапсуляции состояния/повторяемой логики.

- Чистая функция не является composable - нужно состояние
- Возвращать нужно plain object с реактивными переменными

```
<script>
// useCounter.js
import { ref, computed } from 'vue';

export function useCounter() {
  const count = ref(0);
  const doubled = computed(() => count.value * 2);

  function increment() {
    count.value++;
  }

  return { count, doubled, increment };
}
</script>
```

Должен начинаться с use

Еще пара интересных штук

ИТМО

Vue Router

Библиотека маршрутизации.

```
<RouterLink
to="/users">Пользователи</Route
rLink>
<RouterView />
```

Pinia

Библиотека организации общего состояния в рамках приложения.

```
export const useCounterStore =
defineStore('counter', () => {
  const count = ref(0);

  function increment() {
    count.value++;
  }

  return { count, increment };
});
```

Svelte

ИТМО

Что это

Svelte - компилируемый фреймворк для создания веб-интерфейсов. Разработан Ричем Харрисом в 2016 году.

Особенности

- Отсутствие виртуального DOM
- Компонентная архитектура
- Встроенная реактивность
- SvelteKit

Svelte component

ИТМО

```
<!-- Greeting.svelte -->
<script>
  let name = 'Анна';
</script>

<p class="greeting">Привет, {name}!</p>

<style>
  .greeting {
    color: blue;
  }
</style>
```

```
<script>
  import Greeting from './Greeting.svelte';
</script>

<Greeting />
```

Runes

ИТМО

Специальные конструкции, управляющие реактивностью и интерфейсом компонента

Включают:

- $state
- $derived
- $props
- $effect

```
<script>
  let count = $state(0);

  function increment() {
    count += 1;
  }
</script>

<p>Счётчик: {count}</p>
<button onclick={increment}>+1</button>
```

props/effect

ИТМО

```
// Greeting.svelte
<script>
  let { name = 'Гость' } = $props();
</script>

<p>Привет, {name}!</p>

// App.svelte
<script>
  import Greeting from './Greeting.svelte';
</script>

<Greeting name="Анна" />
<Greeting name="Иван" />
<Greeting />
```

```
<script>
  let count = $state(0);

  $effect(() => {
    document.title = `Счётчик: ${count}`;
  });
</script>

<button onclick={() => count++}>
  {count}
</button>
```

Связываемый prop

ИТМО

```svelte
<!-- NameInput.svelte -->
<script>
  let { value = $bindable('') } = $props();
</script>

<input bind:value />
```

Используем с осторожностью

```svelte
<script>
  import NameInput from './NameInput.svelte';

  let name = $state('');
</script>

<NameInput bind:value={name} />
<p>{name}</p>
```

derived

ИТМО

```svelte
<script>
  let count = $state(0);
  let doubled = $derived(count * 2);
</script>

<p>{count}</p>
<p>Удвоенное значение: {doubled}</p>

<button onclick={() => count++}>+1</button>
```

Callback props

ИТМО

```svelte
<!-- SaveButton.svelte -->
<script>
  let { onSave } = $props();
</script>

<button onclick={() => onSave?.('Данные сохранены')}>
  Сохранить
</button>
```

```svelte
<script>
  import SaveButton from './SaveButton.svelte';

  let message = $state('');

  function handleSave(value) {
    message = value;
  }
</script>

<SaveButton onSave={handleSave} />
<p>{message}</p>
```

Snippet

ИТМО

```svelte
<!-- Card.svelte -->
<script>
  let { header, children } = $props();
</script>

<section class="card">
  <header>
    {@render header?.()}
  </header>

  {@render children?.()}
</section>
```

```svelte
<script>
  import Card from './Card.svelte';
</script>

{#snippet cardHeader()}
  <h2>Профиль</h2>
{/snippet}

{#snippet cardContent()}
  <p>Анна, 20 лет</p>
{/snippet}

<Card header={cardHeader} children={cardContent} />
```

Snippet children

ИТМО

```
<Card>
  <p>Содержимое карточки</p>
</Card>

<script>
  let { children } = $props();
</script>

<section>
  {@render children?.()}
</section>
```

Логические блоки

ИТМО

```
<script>
  let loggedIn = $state(false);
</script>

{#if loggedIn}
  <p>Добро пожаловать!</p>
{:else}
  <p>Войдите в систему</p>
{/if}
```

```
{#if role === 'admin'}
  <p>Администратор</p>
{:else if role === 'teacher'}
  <p>Преподаватель</p>
{:else}
  <p>Студент</p>
{/if}
```

```
<script>
  let users = $state([
    { id: 1, name: 'Анна' },
    { id: 2, name: 'Иван' }
  ]);
</script>

<ul>
  {#each users as user (user.id)}
    <li>{user.name}</li>
  {/each}
</ul>
```

Асинхронные значения/bind

ИТМО

```
<script>
  const userPromise = fetch('/api/user')
    .then(response => response.json());
</script>

{#await userPromise}
  <p>Загрузка...</p>
{:then user}
  <p>{user.name}</p>
{:catch error}
  <p>Ошибка: {error.message}</p>
{/await}
```

```
<script>
  let name = $state('');
</script>

<input bind:value={name} />
<p>Привет, {name}!</p>
```

Директивы

ИТМО

bind:	Двусторонняя привязка
use:	Action для DOM-элемента
class:	Условное применение класса
style:	Динамический стиль
transition:	Анимация появления и исчезновения
in: / out:	Отдельные входная и выходная анимации
animate:	Анимация изменения положения

```
<script>
  let active = $state(false);
</script>

<button
  class:active
  onclick={() => active = !active}
>
  Переключить
</button>
```

lifecycle-функции

Включает lifecycle функции:

- onMounted - компонент смонтирован
- tick - Promise, выполняющийся после обновления DOM
- onDestroy - компонент удалён

```
<script>
  import { onMount, onDestroy, tick } from 'svelte';

  onMount(() => {
    console.log('Компонент смонтирован');

    return () => {
      console.log('Очистка');
    };
  });

  onDestroy(() => {
    console.log('Компонент удаляется');
  });
</script>
```

Context API

ИТМО

```svelte
<script>
  import { setContext } from 'svelte';

  setContext('theme', 'dark');
</script>

<slot />
```

```svelte
<script>
  import { getContext } from 'svelte';

  const theme = getContext('theme');
</script>

<p>Тема: {theme}</p>
```

Store

Объект с контрактом подписки, хранящий реактивное состояние вне компонента

writable - объявление изменяемого сигнала

Лучше использовать руны для изоляции состояния

```
<script>
  // counter.js
  import { writable } from 'svelte/store';

  export const count = writable(0);
</script>

<script>
  import { count } from './counter.js';
</script>

<p>{$count}</p>

<button onclick={() => count.update(value => value + 1)}>
  +1
</button>
```

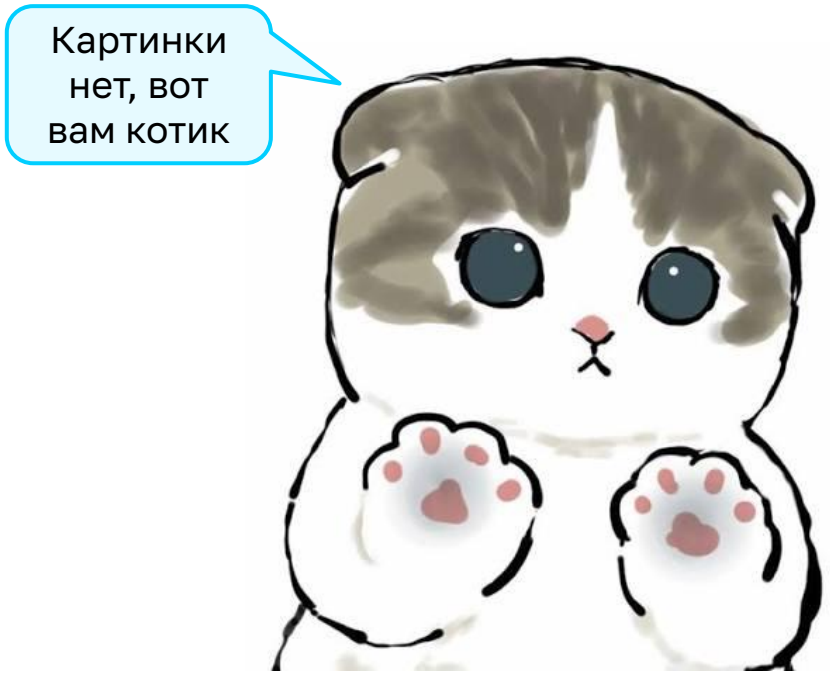
Подписка компонент на store

SvelteKit

ИТМО

Фреймворк для разработки и сборки веб-приложений на базе Svelte. Предоставляет:

- Файловую маршрутизацию
- Серверный рендеринг
- Сборку на Vite
- И много другое...



Спасибо
за внимание!

ITMO more than a UNIVERSITY